



Cyprus international university

CMPE343 Database Management Systems and Programming I (Ercan Airport Management Information System)

PROJECT REPORT

Group Members

NAME - SURNAME	STUDENT NUMBER	GROUP	DEPARTMENT
Eyad Mohamed	22312037	01	CMPE
Ahmed Mohammed	22314055	02	CMPE
Mohamed Al fatih	22308361	02	CMPE

Introduction

- This project presents the design and implementation of a database management system for Ercan Airport. The goal is to create a reliable, organized system to store and manage all airport operations digitally. Currently, this information is likely scattered. This database brings everything together from airplanes and hangars to employees, flights, and passengers into one efficient platform.
- The system keeps track of where airplanes are parked, when they enter or leave hangars, which technicians test which planes, and what scores those tests receive. It also manages employee details, including union memberships and medical exams for traffic controllers. Beyond the basic requirements, the design includes practical features like ticket sales, baggage tracking, and gate assignments to make the database more realistic and useful.

- The final product includes a complete database schema (20 tables), sample data, and 15 management queries that help answer important questions like "Which planes are overdue for maintenance?" or "What is our total ticket revenue per flight?"

ASSUMPTIONS	
SSN	Every employee is uniquely identified by a Social Security Number (SSN).
UNION	All employees (technicians, traffic controllers, etc.) must belong to a union, and each has a unique union membership number
HANGAR	An airplane can be in a hangar for a period of time. The system stores both entry (in_datetime) and exit (out_datetime) times. If a plane is currently inside, the exit time is left empty (NULL)
TEST	All test scores range from 0 to 100
FLIGHTS	A technician can be an expert on multiple plane models, and multiple technicians can be experts on the same model
FLIGHT STATUS	Each flight has a current status (e.g., SCHEDULED, ARRIVED, DEPARTED, CANCELLED) for real-time tracking

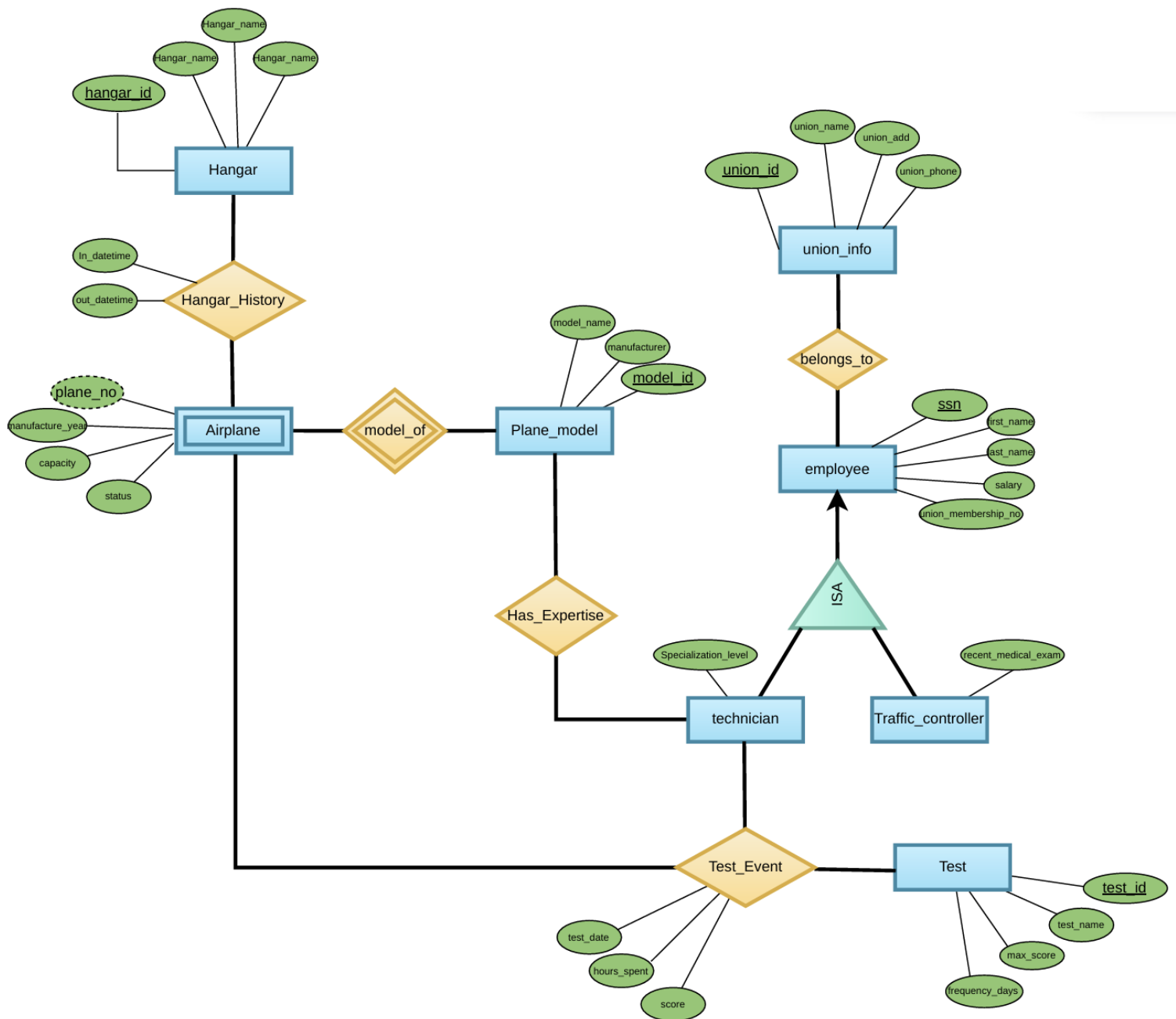
Key References

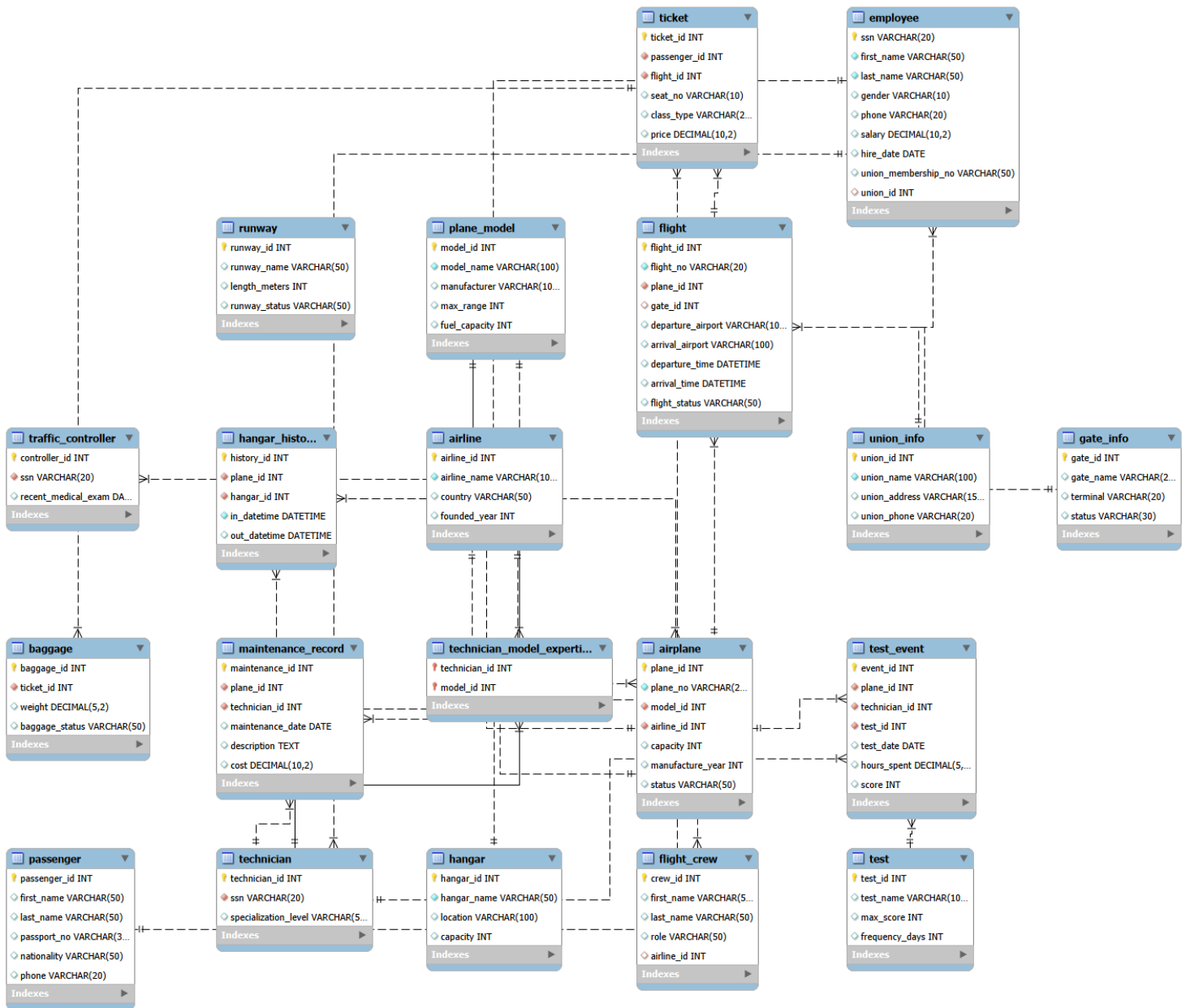
The database consists of 20 tables connected through a total of 19 relationships. The table below highlights the 11 most essential relationships that define the core business logic of the airport management system.

Relationship	Type	Explanation
Airline → Airplane	One-to-Many	One airline owns many airplanes.
Plane_Model → Airplane	One-to-Many	One plane model is used by many airplanes.
Airplane ↔ Hangar	Many-to-Many	An airplane can be stored in many hangars over time, and a hangar can store many airplanes. (Implemented via Hangar_History)
Employee → Technician	One-to-One	An employee can be a technician (specialized role).
Employee → Traffic_Controller	One-to-One	An employee can be a traffic controller (specialized role).
Employee → Union_Info	Many-to-One	Many employees belong to one union.
Technician ↔ Plane_Model	Many-to-Many	A technician can be an expert on many plane models, and a model can have many expert technicians.
Airplane ↔ Test	Many-to-Many	An airplane can undergo many tests, and a test can be performed on many airplanes. (Implemented via Test_Event)
Airplane → Flight	One-to-Many	One airplane can be scheduled for many flights.
Passenger ↔ Flight	Many-to-Many	A passenger can book many flights, and a flight can have many passengers. (Implemented via Ticket)
Ticket → Baggage	One-to-Many	One ticket is linked to exactly one piece of checked baggage.

Entity Relationship (ER) Diagram

The Entity Relationship (ER) diagram provided below illustrates the logical structure of the Ercan Airport Management Database. It serves as a visual blueprint that maps out all 20 entities (tables) and the relationships between them.





Relational Data Model (Schema Layout):

The Relational Data Model translates the abstract conceptual design from the Entity-Relationship Diagram (ERD) into a structured blueprint of normalized tables.

1. **Primary Keys:** Underlined attributes enforce entity integrity, ensuring each record is unique.
 2. **Foreign Keys:** Mapped with arrows →, these keys enforce referential integrity between dependent entities (e.g., linking a flight to its specific plane and gate).
- **Airline** (*airline_id*, airline_name, country, founded_year)
 - **Plane_model** (*model_id*, model_name, manufacturer, max_range, fuel_capacity)
 - **Airplane** (*plane_id*, plane_no, model_id → Plane_Model, airline_id → Airline, capacity, manufacture_year, status)
 - **Hangar** (*hangar_id*, hangar_name, location, capacity)
 - **Hangar_History** (*history_id*, plane_id → Airplane, hangar_id → Hangar, in_datetime, out_datetime)
 - **Union_Info** (*union_id*, union_name, union_address, union_phone)
 - **Employee** (*ssn*, first_name, last_name, gender, phone, salary, hire_date, union_membership_no, union_id → Union_Info)
 - **Technician** (*technician_id*, ssn → Employee, specialization_level)
 - **Traffic_controller** (*controller_id*, ssn → Employee, recent_medical_exam)
 - **Test** (*test_id*, test_name, max_score, frequency_days)
 - **Test_event** (*event_id*, plane_id → Airplane, technician_id → Technician, test_id → Test, test_date, hours_spent, score)

- **Technician_Model_Expertise** (*technician_id* → *Technician*, *model_id* → *Plane_model*)
- **Gate_Info** (*gate_id*, gate_name, terminal, status)
- **Flight** (*flight_id*, flight_no, plane_id → *Airplane*, gate_id → *Gate_info*, departure_airport, arrival_airport, departure_time, arrival_time, flight_status)
- **Runway** (*runway_id*, runway_name, length_meters, runway_status)
- **Maintenance_record** (*maintenance_id*, plane_id → *Airplane*, technician_id → *Technician*, maintenance_date, description, cost)
- **Passenger** (*passenger_id*, first_name, last_name, passport_no, nationality, phone)
- **Ticket** (*ticket_id*, passenger_id → *Passenger*, flight_id → *Flight*, seat_no, class_type, price)
- **Baggage** (*baggage_id*, ticket_id → *Ticket*, weight, baggage_status)
- **Flight_Crew** (*crew_id*, first_name, last_name, role, airline_id → *Airline*)

Data Definition language (DDL):

```
1 CREATE DATABASE ercan_airport_project;
2 USE ercan_airport_project;
3
4
5 -- =====
6 -- 1. AIRLINE
7 -- =====
8 CREATE TABLE Airline (
9     airline_id INT AUTO_INCREMENT PRIMARY KEY,
10    airline_name VARCHAR(100) NOT NULL UNIQUE,
11    country VARCHAR(50),
12    founded_year INT
13 );
14
15 -- =====
16 -- 2. PLANE MODEL
17 -- =====
18 CREATE TABLE Plane_Model (
19     model_id INT AUTO_INCREMENT PRIMARY KEY,
20     model_name VARCHAR(100) NOT NULL,
21     manufacturer VARCHAR(100),
22     max_range INT CHECK (max_range > 0),
23     fuel_capacity INT
24 );
25
26 -- =====
27 -- 3. AIRPLANE
28 -- =====
29 CREATE TABLE Airplane (
30     plane_id INT AUTO_INCREMENT PRIMARY KEY,
31     plane_no VARCHAR(20) NOT NULL UNIQUE,
32     model_id INT NOT NULL,
33     airline_id INT NOT NULL,
34     capacity INT CHECK (capacity > 0),
35     manufacture_year INT,
36     status VARCHAR(50),
37
38     FOREIGN KEY (model_id)
39     REFERENCES Plane_Model(model_id),
40
41     FOREIGN KEY (airline_id)
42     REFERENCES Airline(airline_id)
43 );
```



```

45  -- =====
46  -- 4. HANGAR
47  -- =====
48  CREATE TABLE Hangar (
49      hangar_id INT AUTO_INCREMENT PRIMARY KEY,
50      hangar_name VARCHAR(50) NOT NULL,
51      location VARCHAR(100),
52      capacity INT CHECK (capacity > 0)
53  );
54
55  -- =====
56  -- 5. HANGAR HISTORY
57  -- =====
58  CREATE TABLE Hangar_History (
59      history_id INT AUTO_INCREMENT PRIMARY KEY,
60      plane_id INT NOT NULL,
61      hangar_id INT NOT NULL,
62      in_datetime DATETIME NOT NULL,
63      out_datetime DATETIME,
64
65      FOREIGN KEY (plane_id)
66      REFERENCES Airplane(plane_id),
67
68      FOREIGN KEY (hangar_id)
69      REFERENCES Hangar(hangar_id)
70  );
71
72  -- =====
73  -- 6. UNION INFO
74  -- =====
75  CREATE TABLE Union_Info (
76      union_id INT AUTO_INCREMENT PRIMARY KEY,
77      union_name VARCHAR(100) NOT NULL,
78      union_address VARCHAR(150),
79      union_phone VARCHAR(20)
80  );
81
82  -- =====
83  -- 7. EMPLOYEE
84  -- =====
85  CREATE TABLE Employee (
86      ssn VARCHAR(20) PRIMARY KEY,
87      first_name VARCHAR(50) NOT NULL,
88      last_name VARCHAR(50) NOT NULL,
89      gender VARCHAR(10),
90      phone VARCHAR(20),
91      salary DECIMAL(10,2),
92      hire_date DATE,
93
94      union_membership_no VARCHAR(50) UNIQUE,
95      union_id INT,
96
97      FOREIGN KEY (union_id)
98      REFERENCES Union_Info(union_id)
99  );

```

```

100
101 -- =====
102 -- 8. TECHNICIAN
103 -- =====
104 CREATE TABLE Technician (
105     technician_id INT AUTO_INCREMENT PRIMARY KEY,
106     ssn VARCHAR(20) NOT NULL,
107     specialization_level VARCHAR(50),
108
109     FOREIGN KEY (ssn)
110     REFERENCES Employee(ssn)
111 );
112
113 -- =====
114 -- 9. TRAFFIC CONTROLLER
115 -- =====
116 CREATE TABLE Traffic_Controller (
117     controller_id INT AUTO_INCREMENT PRIMARY KEY,
118     ssn VARCHAR(20) NOT NULL,
119     recent_medical_exam DATE,
120
121     FOREIGN KEY (ssn)
122     REFERENCES Employee(ssn)
123 );
124
125 -- =====
126 -- 10. TEST
127 -- =====
128 CREATE TABLE Test (
129     test_id INT AUTO_INCREMENT PRIMARY KEY,
130     test_name VARCHAR(100),
131     max_score INT,
132     frequency_days INT
133 );
134
135 -- =====
136 -- 11. TEST EVENT
137 -- =====
138 CREATE TABLE Test_Event (
139     event_id INT AUTO_INCREMENT PRIMARY KEY,
140     plane_id INT NOT NULL,
141     technician_id INT NOT NULL,
142     test_id INT NOT NULL,
143     test_date DATE,
144     hours_spent DECIMAL(5,2),
145     score INT CHECK (score BETWEEN 0 AND 100),
146
147     FOREIGN KEY (plane_id)
148     REFERENCES Airplane(plane_id),
149
150     FOREIGN KEY (technician_id)
151     REFERENCES Technician(technician_id),
152
153     FOREIGN KEY (test_id)
154     REFERENCES Test(test_id)
155 );

```

```

157 -- =====
158 -- 12. TECHNICIAN MODEL EXPERTISE
159 -- =====
160 CREATE TABLE Technician_Model_Expertise (
161     technician_id INT,
162     model_id INT,
163
164     PRIMARY KEY (technician_id, model_id),
165
166     FOREIGN KEY (technician_id)
167     REFERENCES Technician(technician_id),
168
169     FOREIGN KEY (model_id)
170     REFERENCES Plane_Model(model_id)
171 );
172
173 -- =====
174 -- 13. GATE INFO
175 -- =====
176 CREATE TABLE Gate_Info (
177     gate_id INT AUTO_INCREMENT PRIMARY KEY,
178     gate_name VARCHAR(20),
179     terminal VARCHAR(20),
180     status VARCHAR(30)
181 );
182
183 -- =====
184 -- 14. FLIGHT
185 -- =====
186 CREATE TABLE Flight (
187     flight_id INT AUTO_INCREMENT PRIMARY KEY,
188     flight_no VARCHAR(20) NOT NULL UNIQUE,
189     plane_id INT NOT NULL,
190     gate_id INT,
191     departure_airport VARCHAR(100),
192     arrival_airport VARCHAR(100),
193     departure_time DATETIME,
194     arrival_time DATETIME,
195     flight_status VARCHAR(50),
196
197     FOREIGN KEY (plane_id)
198     REFERENCES Airplane(plane_id),
199
200     FOREIGN KEY (gate_id)
201     REFERENCES Gate_Info(gate_id)
202 );
203
204 -- =====
205 -- 15. RUNWAY
206 -- =====
207 CREATE TABLE Runway (
208     runway_id INT AUTO_INCREMENT PRIMARY KEY,
209     runway_name VARCHAR(50),
210     length_meters INT,
211     runway_status VARCHAR(50)
212 );
213

```

```

214 -- =====
215 -- 16. MAINTENANCE RECORD
216 -- =====
217 CREATE TABLE Maintenance_Record (
218     maintenance_id INT AUTO_INCREMENT PRIMARY KEY,
219     plane_id INT NOT NULL,
220     technician_id INT NOT NULL,
221     maintenance_date DATE,
222     description TEXT,
223     cost DECIMAL(10,2),
224
225     FOREIGN KEY (plane_id)
226     REFERENCES Airplane(plane_id),
227
228     FOREIGN KEY (technician_id)
229     REFERENCES Technician(technician_id)
230 );
231
232 -- =====
233 -- 17. PASSENGER
234 -- =====
235 CREATE TABLE Passenger (
236     passenger_id INT AUTO_INCREMENT PRIMARY KEY,
237     first_name VARCHAR(50),
238     last_name VARCHAR(50),
239     passport_no VARCHAR(30) UNIQUE,
240     nationality VARCHAR(50),
241     phone VARCHAR(20)
242 );
243
244 -- =====
245 -- 18. TICKET
246 -- =====
247 CREATE TABLE Ticket (
248     ticket_id INT AUTO_INCREMENT PRIMARY KEY,
249     passenger_id INT NOT NULL,
250     flight_id INT NOT NULL,
251     seat_no VARCHAR(10),
252     class_type VARCHAR(20),
253     price DECIMAL(10,2),
254
255     FOREIGN KEY (passenger_id)
256     REFERENCES Passenger(passenger_id),
257
258     FOREIGN KEY (flight_id)
259     REFERENCES Flight(flight_id)
260 );
261

```

```

262 -- =====
263 -- 19. BAGGAGE
264 -- =====
265 CREATE TABLE Baggage (
266     baggage_id INT AUTO_INCREMENT PRIMARY KEY,
267     ticket_id INT NOT NULL,
268     weight DECIMAL(5,2),
269     baggage_status VARCHAR(50),
270
271     FOREIGN KEY (ticket_id)
272     REFERENCES Ticket(ticket_id)
273 );
274
275 -- =====
276 -- 20. FLIGHT CREW
277 -- =====
278 CREATE TABLE Flight_Crew (
279     crew_id INT AUTO_INCREMENT PRIMARY KEY,
280     first_name VARCHAR(50),
281     last_name VARCHAR(50),
282     role VARCHAR(50),
283     airline_id INT,
284
285     FOREIGN KEY (airline_id)
286     REFERENCES Airline(airline_id)
287 );

```

Data Definition Language details:

The DDL script (DDL_ercan_airport_project.sql) creates the database and all tables. Key implementation details include:

- **Data Types:** VARCHAR for strings, INT for numbers, DECIMAL(10,2) for currency, DATETIME for timestamps.
- **Primary Keys:** Auto-incrementing INT keys using AUTO_INCREMENT.
- **Foreign Keys:** FOREIGN KEY ... REFERENCES ensures referential integrity.
- **Constraints:**
 - NOT NULL for mandatory fields.
 - UNIQUE for fields like plane_no, passport_no, and union_membership_no.
 - CHECK for data ranges (e.g., capacity > 0, score BETWEEN 0 AND 100).
 - Composite PRIMARY KEY (technician_id, model_id) for the many-to-many expertise table.

Data Manipulation Language (DML):

```
6      USE ercan_airport_management;
7
8      -- =====
9      -- AIRLINES
10     -- =====
11     INSERT INTO Airline (airline_name, country, founded_year) VALUES
12     ('Turkish Airlines',      'Turkey', 1933),
13     ('Pegasus Airlines',     'Turkey', 1990),
14     ('Qatar Airways',        'Qatar', 1993),
15     ('Emirates',             'UAE', 1985),
16     ('Royal Jordanian',      'Jordan', 1963),
17     ('Saudi Airlines',       'Saudi Arabia', 1945);
18
19     -- =====
20     -- PLANE MODELS
21     -- =====
22     INSERT INTO Plane_Model (model_name, manufacturer, max_range, fuel_capacity) VALUES
23     ('Boeing 737-800',        'Boeing', 5600, 26000),
24     ('Airbus A320neo',        'Airbus', 6300, 27000),
25     ('Boeing 777-300ER',     'Boeing', 13650, 181000),
26     ('Airbus A330-300',      'Airbus', 11750, 139000),
27     ('Boeing 787 Dreamliner', 'Boeing', 14140, 126000),
28     ('ATR72-600',            'ATR', 1528, 5000);
29
30     -- =====
31     -- AIRPLANES
32     -- =====
33     INSERT INTO Airplane
34     (plane_no, model_id, airline_id, capacity, manufacture_year, status)
35     VALUES
36     ('TC-JGR', 1, 1, 189, 2017, 'ACTIVE'),
37     ('TC-SUX', 1, 1, 189, 2015, 'ACTIVE'),
38     ('PC-AEC', 2, 2, 180, 2020, 'ACTIVE'),
39     ('A7-BDA', 3, 3, 396, 2018, 'ACTIVE'),
40     ('A6-EMR', 4, 4, 300, 2019, 'ACTIVE'),
41     ('JY-RJA', 5, 5, 290, 2021, 'ACTIVE'),
42     ('HZ-ASA', 2, 6, 174, 2018, 'MAINTENANCE'),
43     ('TC-PEG', 2, 2, 180, 2019, 'UNDER INSPECTION');
44
45     -- =====
46     -- HANGARS
47     -- =====
48     INSERT INTO Hangar (hangar_name, location, capacity) VALUES
49     ('H-01', 'North Apron Section A', 4),
50     ('H-02', 'North Apron Section B', 6),
51     ('H-03', 'South Maintenance Zone', 3),
52     ('H-04', 'VIP Aircraft Zone', 2);
53
54     -- =====
55     -- HANGAR HISTORY
56     -- =====
57     INSERT INTO Hangar_History
58     (plane_id, hangar_id, in_datetime, out_datetime)
59     VALUES
60     (7, 1, '2026-04-15 08:00:00', NULL),
61     (8, 2, '2026-05-01 10:30:00', NULL),
62     (1, 3, '2026-03-01 07:00:00', '2026-03-10 16:00:00'),
63     (2, 1, '2026-01-05 06:00:00', '2026-01-15 18:00:00'),
64     (4, 4, '2026-02-10 09:00:00', '2026-02-14 13:00:00');
65
```

```

66  -- =====
67  -- UNION INFO
68  -- =====
69  INSERT INTO Union_Info (union_name, union_address, union_phone) VALUES
70  ('Aviation Technicians Union', 'Nicosia, Cyprus', '+90-392-200-1001'),
71  ('Air Traffic Controllers Union', 'Istanbul, Turkey', '+90-212-450-2200'),
72  ('Airport General Workers Union', 'Ankara, Turkey', '+90-312-670-4500'),
73  ('Middle East Aviation Union', 'Doha, Qatar', '+974-4400-8899');
74
75  -- =====
76  -- EMPLOYEES
77  -- =====
78  INSERT INTO Employee
79  (ssn, first_name, last_name, gender, phone, salary,
80  hire_date, union_membership_no, union_id)
81  VALUES
82  ('1001', 'Eyad', 'Mohamed', 'Male', '0533000001', 7200.00, '2018-03-15', 'ATU-001', 1),
83  ('1002', 'Ahmed', 'Yasir', 'Male', '0533000002', 6800.00, '2019-07-12', 'ATU-002', 1),
84  ('1003', 'Mohamed', 'Al-Fatih', 'Male', '0533000003', 9100.00, '2017-01-10', 'ATC-001', 2),
85  ('1004', 'Taha', 'Salah', 'Male', '0533000004', 8500.00, '2020-04-18', 'ATC-002', 2),
86  ('1005', 'Salim', 'Omar', 'Male', '0533000005', 7600.00, '2016-09-25', 'ATU-003', 1),
87  ('1006', 'Ali', 'Essam', 'Male', '0533000006', 7000.00, '2021-06-14', 'AGW-001', 3),
88  ('1007', 'Tariq', 'Kanaan', 'Male', '0533000007', 9500.00, '2015-11-02', 'ATC-003', 2),
89  ('1008', 'Fadi', 'Saleh', 'Male', '0533000008', 7800.00, '2019-12-19', 'ATU-004', 1),
90  ('1009', 'Hassan', 'Darwish', 'Male', '0533000009', 7300.00, '2022-02-10', 'AGW-002', 3),
91  ('1010', 'Samir', 'Al-Masri', 'Male', '0533000010', 8900.00, '2018-08-08', 'MEA-001', 4);
92
93  -- =====
94  -- TECHNICIANS
95  -- =====
96  INSERT INTO Technician (ssn, specialization_level) VALUES
97  ('1001', 'Senior'),
98  ('1002', 'Junior'),
99  ('1005', 'Expert'),
100  ('1008', 'Senior');
101
102  -- =====
103  -- TRAFFIC CONTROLLERS
104  -- =====
105  INSERT INTO Traffic_Controller
106  (ssn, recent_medical_exam)
107  VALUES
108  ('1003', '2025-11-10'),
109  ('1004', '2025-09-20'),
110  ('1007', '2026-01-15'),
111  ('1010', '2025-12-01');
112
113  -- =====
114  -- TEST TYPES
115  -- =====
116  INSERT INTO Test
117  (test_name, max_score, frequency_days)
118  VALUES
119  ('Engine Performance Test', 100, 90),
120  ('Hydraulic System Check', 100, 180),
121  ('Avionics Calibration', 100, 365),
122  ('Structural Inspection', 100, 730),
123  ('Landing Gear Cycle Test', 100, 90),
124  ('Fuel System Leak Test', 100, 180);
125
126  -- =====
127  -- TECHNICIAN MODEL EXPERTISE
128  -- =====
129  INSERT INTO Technician_Model_Expertise
130  (technician_id, model_id)
131  VALUES
132  (1,1),
133  (1,3),
134  (2,2),
135  (2,4),
136  (3,1),
137  (3,5),
138  (4,6);
139

```

```

140  -- =====
141  -- TEST EVENTS
142  -- =====
143  INSERT INTO Test_Event
144  (plane_id, technician_id, test_id,
145  test_date, hours_spent, score)
146  VALUES
147  (1,1,1,'2026-03-11',9.0,92),
148  (1,3,6,'2026-03-12',5.0,88),
149  (7,1,1,'2026-04-15',8.5,62),
150  (7,3,2,'2026-04-15',5.5,78),
151  (5,2,3,'2026-01-14',6.0,90),
152  (4,4,4,'2026-04-05',18.0,75),
153  (2,1,1,'2026-03-25',8.5,80),
154  (8,2,5,'2026-05-01',4.0,70);
155
156  -- =====
157  -- GATE INFO
158  -- =====
159  INSERT INTO Gate_Info
160  (gate_name, terminal, status)
161  VALUES
162  ('G1', 'Terminal 1', 'ACTIVE'),
163  ('G2', 'Terminal 1', 'ACTIVE'),
164  ('G3', 'Terminal 2', 'ACTIVE'),
165  ('G4', 'Terminal 2', 'MAINTENANCE'),
166  ('VIP1', 'VIP Terminal', 'ACTIVE');
167
168  -- =====
169  -- FLIGHTS
170  -- =====
171  INSERT INTO Flight
172  (flight_no, plane_id, gate_id,
173  departure_airport, arrival_airport,
174  departure_time, arrival_time, flight_status)
175  VALUES
176  ('TK1234',1,1,'Ercan','Istanbul','2026-05-18 06:00:00','2026-05-18 07:30:00','ARRIVED'),
177  ('TK1235',1,2,'Ercan','Istanbul','2026-05-19 06:00:00','2026-05-19 07:30:00','SCHEDULED'),
178  ('PC200',3,3,'Ercan','London','2026-05-18 09:30:00','2026-05-18 13:00:00','DEPARTED'),
179  ('QR900',4,4,'Ercan','Doha','2026-05-17 22:00:00','2026-05-18 03:30:00','ARRIVED'),
180  ('RJ101',6,1,'Ercan','Amman','2026-05-18 11:00:00','2026-05-18 12:30:00','SCHEDULED'),
181  ('SV550',7,2,'Ercan','Jeddah','2026-05-20 16:00:00','2026-05-20 19:40:00','CANCELLED');
182
183  -- =====
184  -- RUNWAYS
185  -- =====
186  INSERT INTO Runway
187  (runway_name, length_meters, runway_status)
188  VALUES
189  ('Runway A', 3500, 'ACTIVE'),
190  ('Runway B', 2800, 'UNDER MAINTENANCE'),
191  ('Runway C', 4000, 'ACTIVE');
192
193  -- =====
194  -- MAINTENANCE RECORDS
195  -- =====
196  INSERT INTO Maintenance_Record
197  (plane_id, technician_id, maintenance_date,
198  description, cost)
199  VALUES
200  (7,1,'2026-04-15','Engine vibration inspection and repair',45000.00),
201  (8,2,'2026-05-01','Landing gear hydraulic maintenance',12000.00),
202  (2,1,'2026-01-05','Engine compressor blade replacement',22000.00),
203  (1,3,'2026-03-01','Annual full aircraft inspection',18000.00),
204  (5,2,'2026-01-10','Hydraulic pressure service',6500.00);
205

```



```

206  -- =====
207  -- PASSENGERS
208  -- =====
209  INSERT INTO Passenger
210  (first_name, last_name, passport_no, nationality, phone)
211  VALUES
212  ('Sami', 'Al-Khatib', 'P10001', 'Jordanian', '0556000001'),
213  ('Nader', 'Hammoud', 'P10002', 'Syrian', '0556000002'),
214  ('Rami', 'Yousef', 'P10003', 'Palestinian', '0556000003'),
215  ('Bilal', 'Mansour', 'P10004', 'Lebanese', '0556000004'),
216  ('Karim', 'Salem', 'P10005', 'Egyptian', '0556000005');
217
218  -- =====
219  -- TICKETS
220  -- =====
221  INSERT INTO Ticket
222  (passenger_id, flight_id, seat_no, class_type, price)
223  VALUES
224  (1,1,'12A','Economy',350.00),
225  (2,1,'12B','Economy',350.00),
226  (3,3,'3C','Business',1200.00),
227  (4,4,'1A','First Class',2500.00),
228  (5,5,'15D','Economy',420.00);
229
230  -- =====
231  -- BAGGAGE
232  -- =====
233  INSERT INTO Baggage
234  (ticket_id, weight, baggage_status)
235  VALUES
236  (1,23.5,'LOADED'),
237  (2,18.0,'LOADED'),
238  (3,30.0,'CHECKED-IN'),
239  (4,25.0,'LOADED'),
240  (5,20.0,'PENDING');
241
242  -- =====
243  -- FLIGHT CREW
244  -- =====
245  INSERT INTO Flight_Crew
246  (first_name, last_name, role, airline_id)
247  VALUES
248  ('Wael', 'Al-Rashid', 'Pilot', 1),
249  ('Bassam', 'Khalil', 'Co-Pilot', 1),
250  ('Ziad', 'Hamdan', 'Cabin Crew', 2),
251  ('Amjad', 'Nasser', 'Cabin Crew', 3),
252  ('Raed', 'Al-Fares', 'Pilot', 4);

```

Management Queries:

```
1  -- Query 1: Display all employee full names and their union membership numbers.
2  • SELECT CONCAT(first_name, ' ', last_name) AS full_name, union_membership_no
3  FROM Employee;
4
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
full_name	union_membership_no			
Eyad Mohamed	ATU-001			
Ahmed Yasir	ATU-002			
Mohamed Al-Fatih	ATC-001			
Taha Salah	ATC-002			
Salim Omar	ATU-003			
Ali Essam	AGW-001			
Tariq Kanaan	ATC-003			
Fadi Saleh	ATU-004			
Hassan Darwish	AGW-002			
Samir Al-Masri	MEA-001			

```
7  -- Query 2: Display the first name, last name, and length of the last name for all technicians.
8  • SELECT Employee.first_name, Employee.last_name, LENGTH(Employee.last_name) AS last_name_length
9  FROM Employee
10 JOIN Technician ON Employee.ssn = Technician.ssn;
11
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
first_name	last_name	last_name_length		
Eyad	Mohamed	7		
Ahmed	Yasir	5		
Salim	Omar	4		
Fadi	Saleh	5		

```
12 -- Query 3: Display the plane number and capacity rounded to the nearest hundred.
13 • SELECT Airplane.plane_no, Airplane.capacity, ROUND(Airplane.capacity, -2) AS rounded_capacity
14 FROM Airplane;
15
16
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
plane_no	capacity	rounded_capacity		
TC-JGR	189	200		
TC-SUX	189	200		
PC-AEC	180	200		
A7-BDA	396	400		
A6-EMR	300	300		
JY-RJA	290	300		
HZ-ASA	174	200		
TC-PEG	189	200		

Limit to 1000 rows

```
17
18 -- Query 4: Display the ticket price truncated to no decimal places.
19 • SELECT Ticket.ticket_id, Ticket.price, TRUNCATE(Ticket.price, 0) AS price_no_cents
20 FROM Ticket;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	ticket_id	price	price_no_cents
▶	1	350.00	350
	2	350.00	350
	3	1200.00	1200
	4	2500.00	2500
	5	420.00	420

Limit to 1000 rows

```
22 -- Query 5: Display all employees hired after January 1, 2018.
23 • SELECT Employee.first_name, Employee.last_name, Employee.hire_date
24 FROM Employee
25 WHERE Employee.hire_date > '2018-01-01';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	first_name	last_name	hire_date
▶	Eyad	Mohamed	2018-03-15
	Ahmed	Yasir	2019-07-12
	Taha	Salah	2020-04-18
	Ali	Essam	2021-06-14
	Fadi	Saleh	2019-12-19
	Hassan	Darwish	2022-02-10
	Samir	Al-Masri	2018-08-08

Limit to 1000 rows

```
27 -- Query 6: Display the flight number and departure time, and show the month of departure.
28 • SELECT Flight.flight_no, Flight.departure_time, MONTH(Flight.departure_time) AS departure_month
29 FROM Flight;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	flight_no	departure_time	departure_month
▶	TK1234	2026-05-18 06:00:00	5
	TK1235	2026-05-19 06:00:00	5
	PC200	2026-05-18 09:30:00	5
	QR900	2026-05-17 22:00:00	5
	RJ101	2026-05-18 11:00:00	5
	SV550	2026-05-20 16:00:00	5

Limit to 1000 rows

```

33  -- Query 7: Display the plane number and test score, adding 10 points as a bonus score.
34  • SELECT Airplane.plane_no, Test_Event.score, Test_Event.score + 10 AS bonus_score
35  FROM Test_Event
36  JOIN Airplane ON Test_Event.plane_id = Airplane.plane_id;
37

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [A](#)

	plane_no	score	bonus_score
▶	TC-JGR	92	102
	TC-JGR	88	98
	HZ-ASA	62	72
	HZ-ASA	78	88
	A6-EMR	90	100
	A7-BDA	75	85
	TC-SUX	80	90
	TC-PEG	70	80

Limit to 1000 rows

```

38  -- Query 8: Display all flights along with their assigned gate name, including flights without a gate.
39  • SELECT Flight.flight_no, Flight.flight_status, Gate_Info.gate_name
40  FROM Flight
41  LEFT JOIN Gate_Info ON Flight.gate_id = Gate_Info.gate_id;
42

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [A](#)

	flight_no	flight_status	gate_name
▶	TK1234	ARRIVED	G1
	TK1235	SCHEDULED	G2
	PC200	DEPARTED	G3
	QR900	ARRIVED	G4
	RJ101	SCHEDULED	G1
	SV550	CANCELLED	G2

Limit to 1000 rows

```

45  -- Query 9: Display all gates along with any flights assigned to them, including gates with no flights.
46  • SELECT Gate_Info.gate_name, Flight.flight_no, Flight.flight_status
47  FROM Flight
48  RIGHT JOIN Gate_Info ON Flight.gate_id = Gate_Info.gate_id;
49

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [A](#)

	gate_name	flight_no	flight_status
▶	G1	TK1234	ARRIVED
	G1	RJ101	SCHEDULED
	G2	TK1235	SCHEDULED
	G2	SV550	CANCELLED
	G3	PC200	DEPARTED
	G4	QR900	ARRIVED
	VIP1	NULL	NULL

Limit to 1000 rows

```
52 -- Query 10: Display all flights that have tickets sold (assigned passengers).
53 • SELECT DISTINCT Flight.flight_no, Flight.departure_airport, Flight.arrival_airport
54 FROM Flight
55 INNER JOIN Ticket ON Flight.flight_id = Ticket.flight_id;
56
```

Result Grid

flight_no	departure_airport	arrival_airport
TK1234	Ercan	Istanbul
PC200	Ercan	London
QR900	Ercan	Doha
RJ101	Ercan	Amman

Limit to 1000 rows

```
59 -- Query 11: Find all airplanes that have never undergone any test.
60 • SELECT Airplane.plane_no, Airplane.status
61 FROM Airplane
62 LEFT JOIN Test_Event ON Airplane.plane_id = Test_Event.plane_id
63 WHERE Test_Event.event_id IS NULL;
64
```

Result Grid

plane_no	status
PC-AEC	ACTIVE
JY-RJA	ACTIVE

Limit to 1000 rows

```
66 -- Query 12: Display all flights whose flight number starts with 'TK' and have status 'SCHEDULED'.
67 • SELECT Flight.flight_no, Flight.departure_time, Flight.flight_status
68 FROM Flight
69 WHERE Flight.flight_no LIKE 'TK%' AND Flight.flight_status = 'SCHEDULED';
70
```

Result Grid

flight_no	departure_time	flight_status
TK1235	2026-05-19 06:00:00	SCHEDULED


```
73 -- Query 13: Calculate the average test score for each plane.
74 • SELECT Airplane.plane_no, AVG(Test_Event.score) AS average_test_score
75 FROM Test_Event
76 JOIN Airplane ON Test_Event.plane_id = Airplane.plane_id
77 GROUP BY Airplane.plane_no;
78
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	plane_no	average_test_score
▶	TC-JGR	90.0000
	HZ-ASA	70.0000
	A6-EMR	90.0000
	A7-BDA	75.0000
	TC-SUX	80.0000
	TC-PEG	70.0000

```
81 -- Query 14: Count how many flights each plane has been assigned to.
82 • SELECT Airplane.plane_no, COUNT(Flight.flight_id) AS total_flights
83 FROM Airplane
84 LEFT JOIN Flight ON Airplane.plane_id = Flight.plane_id
85 GROUP BY Airplane.plane_no;
86
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	plane_no	total_flights
▶	A6-EMR	0
	A7-BDA	1
	HZ-ASA	1
	JY-RJA	1
	PC-AEC	1
	TC-JGR	2
	TC-PEG	0
	TC-SUX	0

```
89 -- Query 15: Find the highest paid employee in each union.
90 • SELECT Union_Info.union_name, Employee.first_name, Employee.last_name, Employee.salary
91 FROM Employee
92 JOIN Union_Info ON Employee.union_id = Union_Info.union_id
93 WHERE (Employee.union_id, Employee.salary) IN (
94     SELECT Employee.union_id, MAX(Employee.salary)
95     FROM Employee
96     WHERE Employee.union_id IS NOT NULL
97     GROUP BY Employee.union_id
98 );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	union_name	first_name	last_name	salary
▶	Aviation Technicians Union	Fadi	Saleh	7800.00
	Air Traffic Controllers Union	Tariq	Kanaan	9500.00
	Airport General Workers Union	Hassan	Darwish	7300.00
	Middle East Aviation Union	Samir	Al-Masri	8900.00

